



Symfony



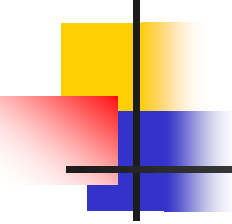
3. PHP – Características básicas del lenguaje



PhpStorm

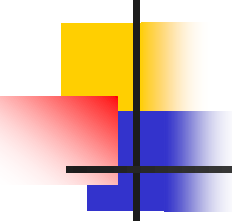
DWES

Profesor: Alejandro Amat Reina



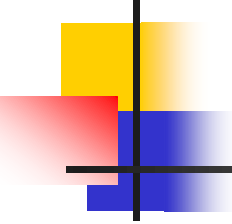
Comenzamos desde cero

- *Crea un documento html (index.html) con un párrafo que muestre el texto «Hola mundo!!».*
- *Muéstralo con el navegador.*



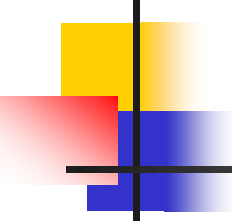
Extensión php

- *Renombra el documento index.html a index.php y vuelve a mostrarlo con el navegador.*
- *¿Hay alguna diferencia en la visualización del documento?*
- *¿Y en la forma que el servidor lo procesa?*



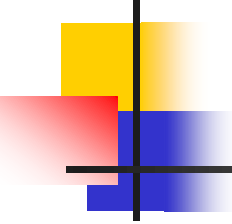
Nuestro primer código PHP

- El código php siempre va entre los simbolos `<?php` y `?>`.
- Las instrucciones php terminan siempre con ;
- Para generar código html desde php podemos utilizar el método echo pasándole el texto del código que queremos generar.
- *Sustituye el texto que hay dentro del párrafo por un bloque de código php que muestre el mismo texto.*
- *Vuelve a mostrarlo con el navegador.*



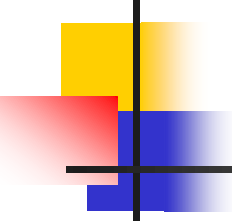
Variables en PHP

- Los nombres de las variables siempre comienzan por \$
- Después del \$ los nombres de las variables deben ir seguidos por una letra o el carácter _ y pueden contener también números.
- No es necesario declarar una variable ni especificarle un tipo (entero, cadena,...) concreto.
- *Crea una variable que contenga el texto a generar y utiliza la función echo para mostrar su contenido.*



Código PHP en cualquier lugar

- *En el documento index.php crea la variable en un bloque PHP al principio del todo (antes del DOCTYPE).*
- *Crea otro bloque de código en el elemento <title> que muestre el contenido de la variable como título de la página.*
- *Finalmente, crea otro bloque de código PHP que muestre el contenido de la variable en el párrafo del body.*



Tipos de las variables

- El tipo de la variable se decide en función del contexto en que se emplee.

// Al asignarle el valor 7, la variable es de tipo "entero"

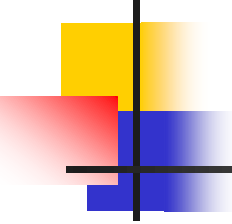
```
$mi_variable = 7;
```

// Si le cambiamos el contenido

```
$mi_variable = "siete";
```

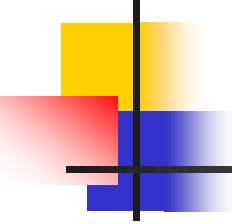
// La variable puede cambiar de tipo

// En este caso pasa a ser de tipo "cadena"



Tipos de datos en PHP

- **booleano** (boolean). Sus posibles valores son true y false. Además, cualquier número entero se considera como true, salvo el 0 que es false.
- **entero** (integer). Cualquier número sin decimales. Se pueden representar en formato decimal, octal (comenzando por un 0), o hexadecimal (comenzando por 0x).
- **real** (float). Cualquier número con decimales. Se pueden representar también en notación científica.
- **cadena** (string). Conjuntos de caracteres delimitados por comillas simples o dobles.
- **null**. Es un tipo de datos especial, que se usa para indicar que la variable no tiene valor.
(<http://php.net/manual/es/language.types.null.php>)



Operadores Aritméticos

Addition

php >

5 + 5



10

Multiplication

php >

5 * 2



10

Subtraction

php >

5 - 2



3

Division

php >

2 / 5



0.4

Exponent

php >

5 ** 2

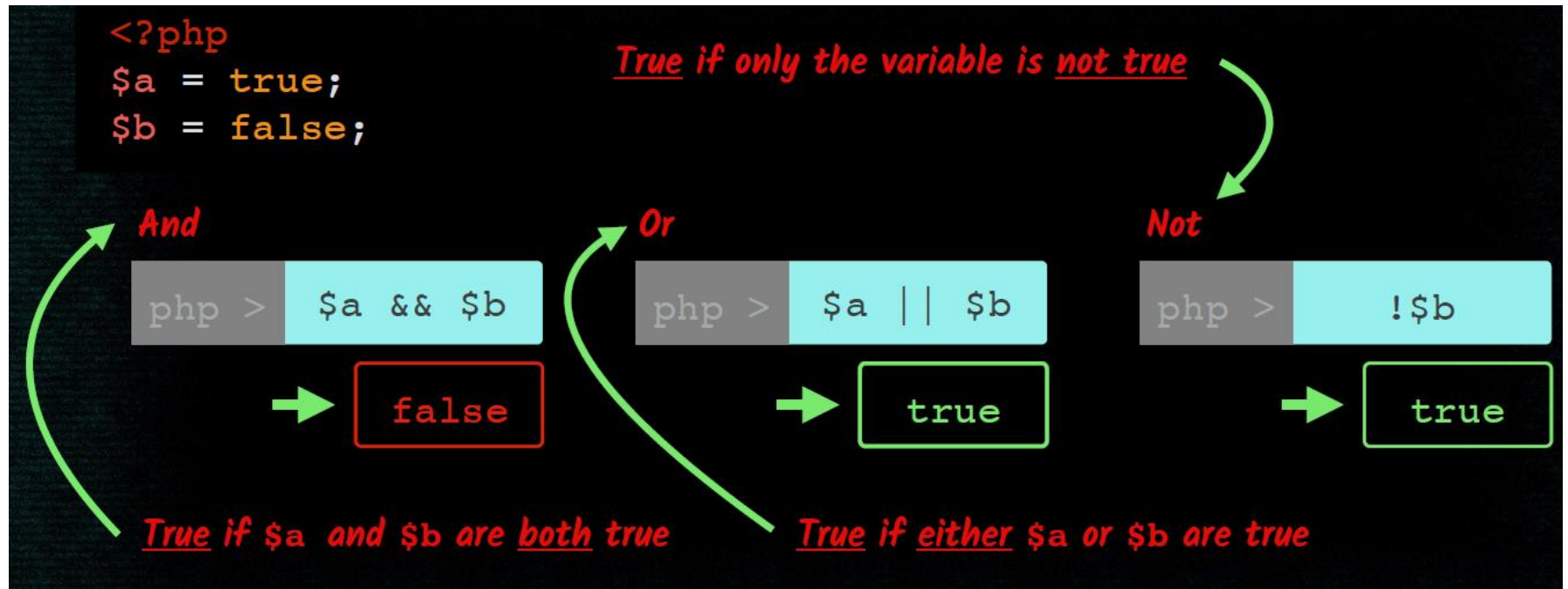


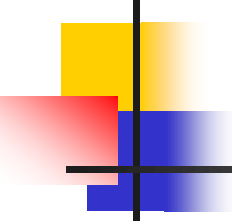
25

Operadores de comparación

Equal php > 5 == '5' → true	Greater Than or Equal To php > 5 >= 5 → true	Not Equal To php > 5 != 2 → true
Greater Than php > 5 > 2 → true	Less Than php > 2 < 5 → true	Identical php > 5 === '5' → false

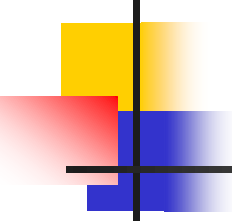
Operadores lógicos





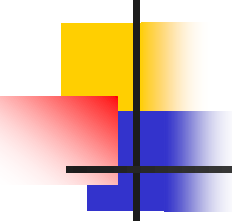
Instrucciones de control de flujo

- Las instrucciones de control de flujo en PHP funcionan exactamente igual que en otros lenguajes ya estudiados por el alumno: java, javascript, C#, etc.
- Las instrucciones de control de flujo más habituales son:
 - Condicionales: if, if else, switch
 - Bucles: while, do while, for



Cadenas de texto

- Podemos usar tanto comillas simples como comillas dobles
- Para concatenar cadenas utilizaremos el operador punto (.)
- *Vamos a cambiar el contenido de nuestra variable por nuestro nombre.*
- *Ahora en el párrafo de la página mostraremos el texto «Bienvenido: Álex»*

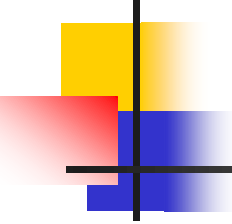


Expansión de variables

- Podemos introducir una variable dentro de un texto siempre y cuando usemos las comillas dobles para delimitar el texto.
- Esto hará que el contenido de la variable se expanda y se concatene con el texto existente en la cadena.
 - `echo "<p>Módulo: $modulo</p>"`
- A veces, es necesario rodearla entre llaves
 - `echo "<p>Módulo: ${modulo}DAW</p>«`
- Si no pusiéramos las llaves el intérprete buscaría una variable que se llame `$moduloDAW`
- *Utiliza la expansión de variables para mostrar el texto de Bienvenida.*

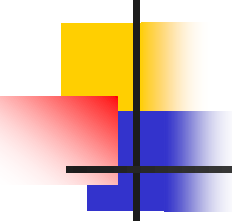


- Nos permiten introducir caracteres especiales dentro de cadenas.
- Se introducen por medio del carácter \
- Con comillas simples sólo podemos usar \' y \\'
- Con comillas dobles tenemos más opciones



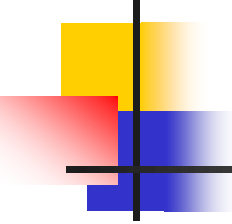
Funciones de cadenas

- No tiene sentido enumerarlas una a una.
- Puedes consultarlas en <http://es.php.net/manual/es/ref.strings.php>



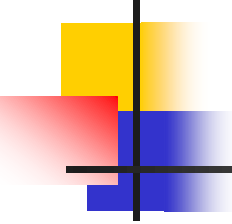
Ejercicios cadenas 1

- *Crea una página llamada cadenas.php, copia el contenido de la página index.php y muestra lo siguiente:*
 - *Elimina los espacios del principio y el final del nombre si los hubiera.*
 - *Elimina los caracteres '/' del principio y el final del nombre si los hubiera.*
 - *La variable nombre en mayúsculas, minúsculas y con la primera letra en mayúscula y las demás en minúsculas.*
 - *Muestra el código ascii de la primera letra del nombre.*
 - *Muestra la longitud del nombre.*
 - *Muestra el número de veces que aparece la letra a (mayúscula o minúscula).*
 - *Muestra la posición de la primera a existente en el nombre (sea mayúscula o minúscula). Si no hay ninguna mostrará -1.*
 - *Lo mismo, pero con la última a.*
 - *Muestra el nombre sustituyendo las letras o por el número cero (sea mayúscula o minúscula).*
 - *Indica si el nombre comienza por 'al' o no.*



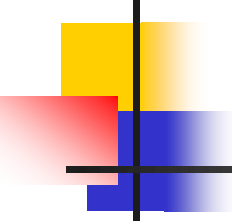
Ejercicios cadenas 2

- A partir de una variable que contenga una url:
 - `$url = 'http://username:password@hostname:9090/path?arg=value#anchor';`
- Utiliza la función `parse_url` para extraer de la url las siguientes partes:
 - El protocolo utilizado (en el ejemplo http).
 - El nombre de usuario (en el ejemplo username).
 - El path de la url (en el ejemplo /path)
 - El querystring de la url (en el ejemplo arg=value)



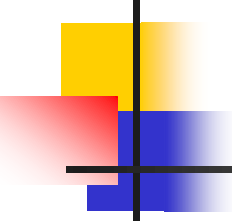
Fechas

- En PHP las fechas se almacenan como números.
- Se utiliza la clase DateTime para representarlas
- Al igual que ocurre con las cadenas no tiene sentido estudiar las funciones relacionadas una a una.
- Las puedes consultar en:
<http://php.net/manual/es/ref.datetime.php>



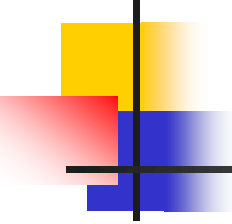
Ejercicios fechas

- *Crea una página llamada fechas.php, copia el contenido de la página index.php y muestra lo siguiente:*
 - *Muestra la fecha y hora actuales con el formato:*
 - *dd/mm/yyyy hh:mm:ss*
 - *Muestra el nombre de la zona horaria que se utiliza por defecto.*
 - *Muestra la fecha de dentro de 45 días.*
 - *Muestra el número de días que han pasado desde el 1 de enero.*
 - *Muestra la fecha y hora actuales de Nueva York.*
 - *Muestra el día de la semana que era el 1 de enero de este año.*



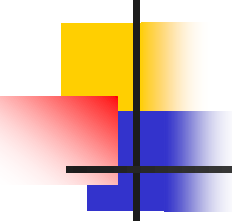
Arrays

- Un array es un tipo de datos que nos permite almacenar varios valores
- Para acceder a un valor utilizaremos una clave.
- Las claves pueden ser números o textos (arrays asociativos).
- Si no indicamos ninguna clave, a cada elemento se le asociará una clave numérica correlativa.



Crear array

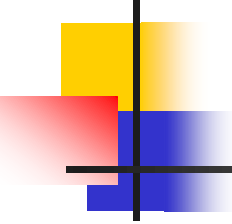
- Podemos crear un array de dos formas:
 - Usando la función array:
`$nombres = array();`
 - Usando un atajo disponible desde la versión 5.4 de PHP:
`$nombres = [];`
 - En ambos ejemplos estamos creando un array vacío.



Crear array con valores

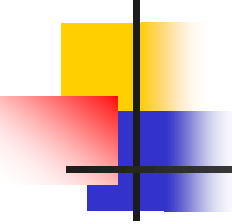
- Si queremos asignar valores al array en el momento de crearlo:

```
$nombres = array('Sandra', 'Pedro', 'Andrea');  
$nombres = ['Sandra', 'Pedro', 'Andrea'];
```



Mostrar el contenido de un array

- Si intentamos mostrar el contenido de un array con `echo` obtendremos el texto `Array`.
- `Echo` no muestra los datos que hay dentro del array.
- En lugar de esto podemos utilizar la función `print_r`:
 - `print_r($nombres);`
- Si necesitamos darle formato a los contenidos del array, tendremos que recorrerlo con un bucle e ir mostrando elemento a elemento.



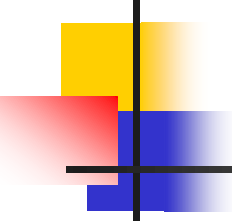
Añadiendo elementos al array

- Podemos añadir elementos al array utilizando corchetes vacíos:

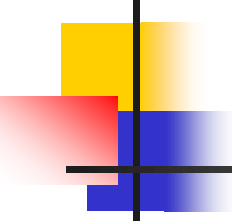
```
$nombres[] = 'Raul';  
$nombres[] = 'Marta';
```

- La clave de estos elementos será el siguiente índice numérico disponible.

Acceder a los elementos del array



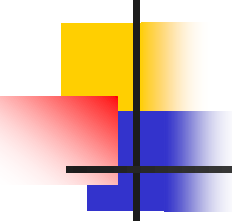
- Accederemos a los elementos del array indicando la clave del elemento entre corchetes:
 - **echo** \$nombres[2];
- Podemos modificar su valor:
 - \$nombres[0] = 'Sara';



Arrays asociativos

- Si el array contiene datos diversos y/o nos interesa acceder a ellos con claves más específicas que un simple índice numérico podemos utilizar arrays asociativos:

```
$alumno = array(  
    'nombre' => 'Sara',  
    'apellido' => 'García',  
    'edad' => 22  
);  
$alumno = [  
    'nombre' => 'Sara',  
    'apellido' => 'García',  
    'edad' => 22  
];  
$alumno['nombre'] = 'Verónica';  
echo $alumno['nombre'];
```



Arrays multidimensionales

- Imagina que queremos tener un array cuyos elementos son nombres de deportes.
- Queremos dividirlos en deportes de invierno y deportes de verano.
- Podemos crear un array `$deportes` que contenga dos elementos que a su vez también serán arrays.

```
$deportes = array(  
    'invierno' => array('esquí de fondo', 'hockey sobre hielo'),  
    'verano' => array('natación', 'voley playa')  
);
```

- Para acceder al primer deporte de invierno:
 - `echo $deportes['invierno'][0];`

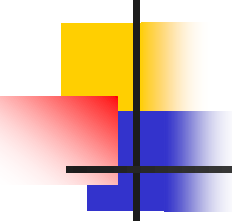
Arrays Multidimensionales asociativos

- La segunda dimensión del array también puede ser un array asociativo:

```
$alumnos = array(  
    array(  
        'nombre' => 'Sara',  
        'edad' => 25  
    ),  
    array(  
        'nombre' => 'Pedro',  
        'edad' => 23  
    ),  
);
```

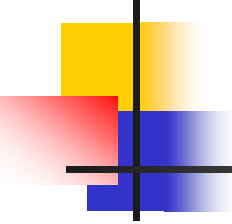
- Mostraremos el nombre del primer alumno:

- `echo $alumnos[0]['nombre'];`



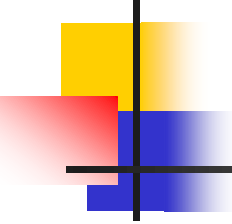
Funciones de arrays

- Como siempre no tiene sentido ir viéndolas de una en una.
- Podemos consultarlas en <http://es.php.net/manual/es/ref.array.php>



Ejercicios de arrays

- *Crea una página llamada arrays.php, copia el contenido de la página index.php y resuelve los ejercicios siguientes utilizando funciones de arrays:*
 - Crea un array con los nombres de varios alumnos de la clase incluyendo el tuyo.
 - Muestra el número de elementos que tiene el array.
 - Crea una cadena que contenga los nombres de los alumnos existentes en el array separados por un espacio y muéstrala.
 - Muestra el array en un orden aleatorio distinto al que lo creaste.
 - Muestra el array ordenado alfabéticamente.
 - Muestra los alumnos cuyo nombre contenga al menos una 'a'.
 - Muestra el array en el orden inverso al que se creó.
 - Muestra la posición que tiene tu nombre en el array.



Recorrer arrays

```
<?php
$meteors = array(
    'Hoba',
    'Cape York',
    'Campo del Cielo',
    'Canyon Diablo',
);

foreach($meteors as $meteor) {
    echo $meteor;
}
```

On each pass through our foreach loop, the data in \$meteor will update with the next item in the collection.

Output

**Hoba
Cape York
Campo del Cielo
Canyon Diablo**

The value, our meteorite names

Recorrer arrays asociativos

```
<?php
$meteors = array(
    'Hoba' => 600000000,
    'Cape York' => 58200000,
    'Campo del Cielo' => 50000000,
    'Canyon Diablo' => 30000000,
);

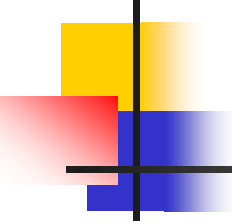
foreach($meteors as $meteor) {
    echo $meteor;
}
```

Output



600000000
58200000
50000000
30000000

The value is our meteorite weight!



Recorrer claves y valores

```
<?php
$meteors = array(
    'Hoba' => 6000000000,
    'Cape York' => 582000000,
    'Campo del Cielo' => 500000000,
    'Canyon Diablo' => 300000000,
);

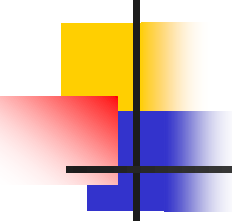
foreach($meteors as $name => $weight){
    echo "$name weighs $weight grams.";
}
```

Output

**Hoba weighs
6000000000 grams.**

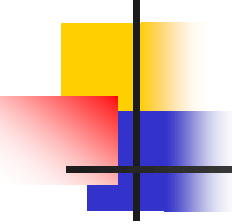
...

**Canyon Diablo weighs
300000000 grams.**



Ejercicios de arrays

- *Crea una página llamada arrays2.php, copia el contenido de la página index.php y resuelve los ejercicios siguientes utilizando funciones de arrays:*
 - *Crea un array de alumnos donde cada elemento sea otro array que contenga el id, nombre y edad del alumno.*
 - *Crea una tabla en la que se muestren todos los datos de los alumnos.*
 - *Utiliza la función array_column para obtener un array indexado que contenga únicamente los nombres de los alumnos y muéstralo por pantalla.*
 - *Crea un array con 10 números :*
 - *utiliza la función array_sum para obtener la suma de los 10 números.*
 - *utiliza la función array_reduce para obtener el factorial de los 10 números.*



Funciones propias

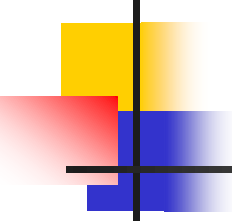
- Para crear tus propias funciones, deberás usar la palabra `function`:

```
function suma($a, $b)
{
    return $a + $b;
}
```

- Para invocar la función:

```
$resultado = suma(5, 7);
```

- Si una función no tiene una sentencia `return`, devuelve `null` al finalizar.

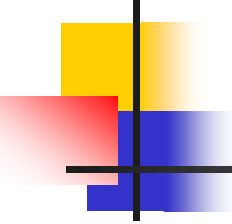


Valores por defecto en los parámetros

- Podemos indicar valores por defecto para los parámetros
- Si cuando llamamos a la función no indicamos el valor de un parámetro se tomará el valor por defecto indicado.

```
function precio_con_iva($precio, $iva=0.21)
{
    return $precio * (1 + $iva);
}
$precio = 10;
$precio_iva = precio_con_iva($precio);
```

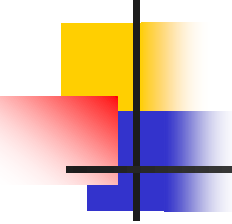
- Puede haber más de un parámetro con valor por defecto, pero siempre tienen que estar al final.



Paso de parámetros por referencia

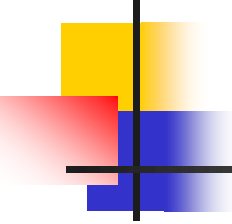
- Por defecto los parámetros se pasan por valor.
- Para pasar un parámetro por referencia añadiremos el símbolo & delante de su nombre.

```
function precio_con_iva(&$precio, $iva=0.18)
{
    $precio *= (1 + $iva);
}
```



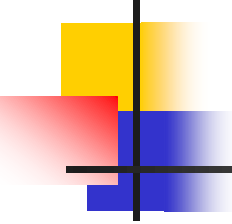
Declaraciones de tipo (Type Hinting)

- Las funciones obligan a que los parámetros sean de cierto tipo
- Si el valor dado es de un tipo incorrecto, se generará un error
- Debe anteponerse el nombre del tipo al nombre del parámetro
- Se puede hacer que una declaración acepte valores NULL si el valor predeterminado del parámetro se establece a NULL



Tipos válidos

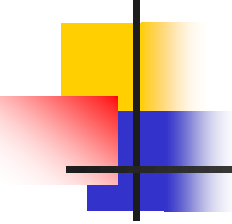
Tipo	Descripción	Versión de PHP mínima
nombre de clase/interfaz	El parámetro debe ser una <i>instanceof</i> del nombre de la clase o interfaz dada.	PHP 5.0.0
<i>self</i>	El parámetro debe ser una <i>instanceof</i> de la misma clase donde está definido el método. Esto solamente se puede utilizar en clases y métodos de instancia.	PHP 5.0.0
array	El parámetro debe ser un array .	PHP 5.1.0
callable	El parámetro debe ser un callable válido.	PHP 5.4.0
bool	El parámetro debe ser un valor de tipo boolean .	PHP 7.0.0
float	El parámetro debe ser un número de tipo float .	PHP 7.0.0
int	El parámetro debe ser un valor de tipo integer .	PHP 7.0.0
string	El parámetro debe ser un string .	PHP 7.0.0



Ejemplo

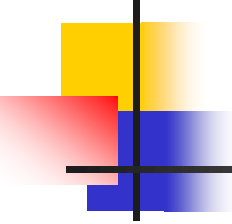
```
function suma(int $a, int $b)
{
    return $a + $b;
}
```

```
$resultado = suma(5, 3);
```



Declaraciones de tipo de devolución

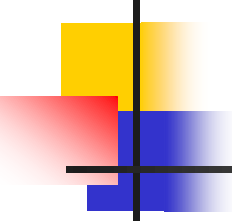
- Añadido en PHP 7
- Especifican el tipo del valor que serán devuelto desde una función
- Están disponibles los mismos tipos de la tabla anterior



Ejemplo

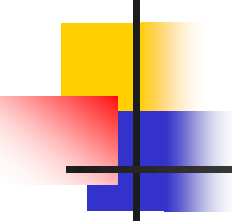
```
function suma(int $a, int $b) : int  
{  
    return $a + $b;  
}
```

```
$resultado = suma('5', 3);
```



Tipificación estricta

- PHP fuerza a los valores de un tipo erróneo a ser del tipo escalar esperado si es posible
- Ejemplo:
 - Una función que espera un string y recibe un int obtendrá una variable de tipo string
- En el modo estricto esto no se permite
- Si los tipos no coinciden nos dará error
- Excepción: se puede pasar un int a una función que espere un float
- El modo estricto se habilita para cada fichero php
- Habilitar el modo estricto:
declare(strict_types=**true**);

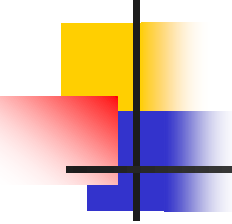


Ejemplo

```
declare(strict_types=true);
```

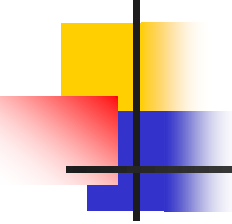
```
function suma(int $a, int $b) : int  
{  
    return $a + $b;  
}
```

```
$resultado = suma('5', 3);
```



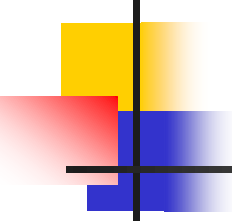
Ejercicio

- Queremos crear una función llamada insert que nos genere una sentencia insert into en sql.
- Para ello la función recibirá dos parámetros:
 - El nombre de la tabla
 - Un array asociativo que contendrá los nombres y valores de los campos de la tabla.
- La sentencia resultante tendrá la siguiente forma:
 - ``insert into nombre_tabla (nombres campos separados por comas) values (valores campos separados por comas con el carácter `:`' delante)`
- De momento no haremos nada con los valores de los campos.
- Ayuda:
 - utiliza las funciones `sprintf`, `implode` y `array_keys`.



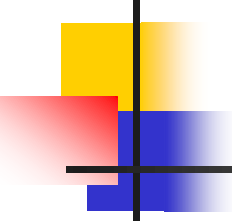
Ejercicio

- A partir del ejercicio anterior crea otra función que reciba los mismos parámetros más un parámetro booleano para indicar si queremos generar la query con los nombres de los campos o no.
- El parámetro tendrá el valor true por defecto.
- Si su valor es true generará la query igual que en el ejercicio anterior, pero si es false la generará así:
 - ``insert into nombre_tabla values (valores campos separados por comas con el carácter `:` delante)`



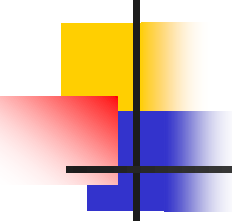
Ejercicio

- Repite el ejercicio anterior con los siguientes cambios:
 - La cadena resultante se pasará por referencia.
 - Pasaremos la cadena de la siguiente forma:
 - insert into tabla (campos) values (valores)
 - Dentro de la función sustituiremos lo siguiente:
 - El texto tabla por el nombre de la tabla.
 - El texto campos por los nombres de los campos separados por comas
 - El texto valores por los nombres de los campos separados por comas y el carácter ':' delante.



Funciones anónimas (closures)

- Están implementadas usando la clase Closure
- Permiten la creación de funciones que no tienen un nombre específico
- Podemos asignar la función a una variable o pasarla como parámetro a otra función



Ejemplo

- Sin parámetros:

```
$anonima = function () {  
    echo "Hola";  
};  
$anonima();
```

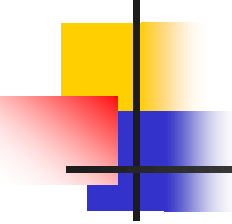
- Con parámetros:

```
$anonima = function ($nombre) {  
    echo "Hola {$nombre}";  
};  
$anonima('Álex');
```

Usar variables del ámbito superior

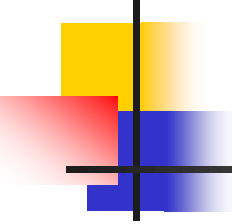
- Una función anónima puede usar variables del ámbito superior mediante la palabra reservada use:

```
function saluda(callable $fnSaluda) {  
    $fnSaluda('Álex');  
}  
$saludo = 'Hola';  
$anonima = function ($nombre) use ($saludo) {  
    echo "{$saludo} {$nombre}";  
};  
saluda($anonima);
```



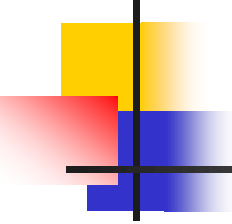
Ejercicio

- Queremos crear una función que muestre el resultado de una operación con dos operandos
- Se mostrará por pantalla el primer operando, el símbolo de la operación, el segundo operando, el símbolo = y el resultado de la operación
- La operación podría ser: suma, resta, multiplicación, etc.
- Ejemplo de salida: $5 + 3 = 8$
- La idea es que la función reciba como parámetro un closure que se encargue de realizar la operación, de forma que podamos llamar distintas veces a la misma función cambiando el closure para distintas operaciones.



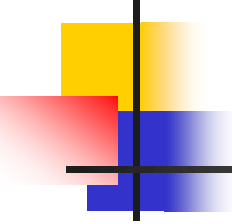
Variables superglobales

- Variables internas predefinidas de PHP
- Pueden usarse desde cualquier ámbito
- Son arrays asociativos que contienen un conjunto de valores
 - `$_SERVER`. Contiene información sobre el entorno del servidor web y de ejecución.
 - `$_GET`, `$_POST` y `$_COOKIE` contienen las variables que se han pasado al script actual utilizando, respectivamente, los métodos GET (parámetros en la URL), HTTP POST y Cookies HTTP
 - `$_REQUEST` junta en uno solo el contenido de los tres arrays anteriores, `$_GET`, `$_POST` y `$_COOKIE`.
 - `$_ENV` contiene las variables que se puedan haber pasado a PHP desde el entorno en que se ejecuta.
 - `$_FILES` contiene los ficheros que se puedan haber subido al servidor utilizando el método POST.
 - `$_SESSION` contiene las variables de sesión disponibles para el guion actual.
- <http://es.php.net/manual/es/language.variables.superglobals.php>



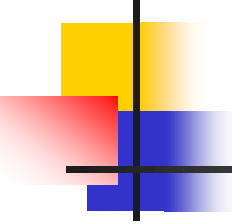
Ejercicio

- Crea una función que devuelva la uri de la página actual eliminando el carácter '/' inicial.



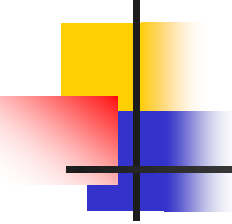
Ejercicio

- Crea una función como la anterior que elimine el querystring de la uri si lo tuviera.



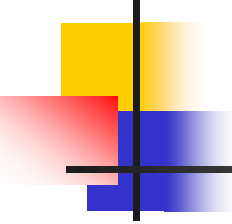
Ejercicio

- Crea una función que nos devuelva el método por el que se ha solicitado la página actual (GET, POST, etc.).



isset y unset

- `isset` determina si una variable o variables están definidas y no son `NULL`
- `unset` destruye las variables especificadas
- Si una variable ha sido eliminada con `unset()`, `isset()` devolverá `FALSE`
- `isset` devolverá `false` si la variable ha sido definida como `NULL`
- Si una variable contiene una cadena vacía `isset` devolverá `true`
- Si son pasados varios parámetros, `isset()` devolverá `true` únicamente si todos los parámetros están definidos
- `isset` se utiliza mucho para comprobar si se ha recibido un parámetro por `GET`



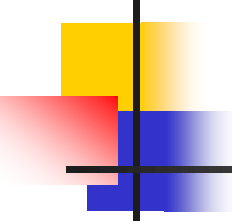
Nuevos operadores de PHP 7

- Operador nave espacial `<=>` (Spaceship operator)

`($a <=> $b) -> ($a < $b) ? -1 : (($a > $b) ? 1 : 0)`

- Operador de fusión de null `??`

`$a = $b ?? "default_value"; -> $a = isset($b) ? $b : "default_value";`



Ejercicio

- Crea una página que compruebe si ha recibido un parámetro llamado número
- En caso de haberlo recibido mostrará por pantalla si el número es mayor, menor o igual que 10
- NOTA: Utiliza los nuevos operadores de PHP 7 `<=>` y `??`